Análisis de algoritmos

I. DATOS INFORMATIVOS

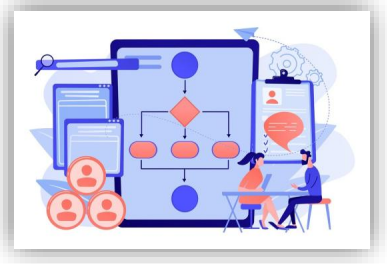
- **Curso** Análisis de algoritmos y Estrategias de Programación
- **Título de la actividad** Desarrollo de casos
- **Tipo de evaluación** Práctica de campo – semana 4
- **Tipo de participación** Grupal
- **Semana de entrega** Disponible hasta la semana 4
- **Medio de presentación** Aula virtual

II. LOGRO DE LA PRÁCTICA

Al finalizar la práctica, el estudiante implementa programas en Python utilizando algoritmos de backtracking para resolver problemas computacionales de forma clara y ordenada.

III. INDICACIONES

- Resolver todos los ejercicios utilizando Python.
- Utilizar comentarios para explicar el funcionamiento del código.
- Ejecutar correctamente cada programa.
- Mostrar evidencias de ejecución.
- Analizar los resultados obtenidos.



IV. ACTIVIDAD

Problema 1: Resolución de Sudoku 6x6/9x9 con visualización paso a paso

Enunciado:

Implemente un programa que resuelva un tablero de Sudoku (6x6 o 9x9, a elección) utilizando el algoritmo de backtracking. El programa debe mostrar en una interfaz gráfica el tablero inicial (con celdas fijas resaltadas) y animar el proceso de resolución, mostrando en tiempo real cómo el algoritmo prueba valores, retrocede al encontrar conflictos y finalmente llega a la solución. Debe incluir un contador de retrocesos (backtracks) realizados y el tiempo total de ejecución.

Librerías sugeridas:

- Opción A: tkinter (para el tablero y grilla) + time (para controlar la velocidad de animación)
- Opción B: pygame (para renderizado gráfico y animación fluida de celdas)

Problema 2: N-Reinas con visualización del árbol de decisiones

Enunciado:

Desarrolle una aplicación que resuelva el problema de las N-Reinas (N ingresado por el usuario, entre 4 y 10) mediante backtracking. La interfaz debe permitir al usuario ingresar el valor de N, mostrar el tablero de ajedrez con las reinas ubicadas en cada solución encontrada, y permitir navegar entre todas las soluciones posibles (anterior/siguiente). Adicionalmente, debe mostrarse un panel con

estadísticas: número total de soluciones, nodos explorados y nodos podados.

Librerías sugeridas:

- Opción A: PyQt5 o PySide6 (para una interfaz con controles interactivos como botones y spinners)
- Opción B: tkinter + matplotlib embebido (para graficar el tablero y las estadísticas)

Problema 3: Laberinto — búsqueda de camino con restricciones (Rat in a Maze)

Enunciado:

Diseñe un programa que, dado un laberinto representado como una matriz (con celdas libres, bloqueadas y una entrada/salida), encuentre todos los caminos posibles desde el punto de inicio hasta el destino usando backtracking, respetando restricciones de movimiento (por ejemplo, no se puede pasar dos veces por la misma celda). La interfaz gráfica debe permitir al usuario diseñar su propio laberinto haciendo clic en las celdas (libre/bloqueada), y luego visualizar animadamente la exploración del algoritmo, resaltando en un color las celdas visitadas y en otro color el camino final encontrado.

Librerías sugeridas:

Opción A: pygame (ideal para dibujar la grilla interactiva y animar la exploración celda por celda)

Opción B: tkinter con Canvas (para dibujar la matriz y actualizar colores dinámicamente)

Problema 4: Coloreado de grafos (Graph Coloring) con selección interactiva de colores

Enunciado:

Implemente una solución que, dado un grafo (definido por el usuario mediante nodos y aristas) y un número máximo de colores disponibles (k), determine mediante backtracking si es posible colorear el grafo de modo que ningún par de nodos adyacentes tenga el mismo color. La interfaz debe permitir construir el grafo de forma interactiva (agregar nodos con clic, conectar aristas), ingresar el valor de k, y visualizar el proceso de asignación y retroceso de colores en cada nodo durante la ejecución del algoritmo.

Librerías sugeridas:

Opción A: networkx (para el modelado del grafo) + matplotlib (para la visualización y actualización del dibujo)

Opción B: PyQt5/PySide6 con QGraphicsView (para una interfaz interactiva de construcción y coloreado de grafos)

V. EVIDENCIA

- Código fuente del proyecto de software solicitado en el apartado IV, incluyendo interfaz gráfica para visualizar las soluciones encontradas.

VI. ENTREGABLE

Informe en un sólo archivo PDF, conteniendo en la primera hoja los miembros del equipo de trabajo que participaron en la actividad, y en las hojas sucesivas el detalle del trabajo realizado que comprende:

FECHA LÍMITE DE PRESENTACIÓN: Domingo 08 de noviembre del 2026

